

Trabalho I — Máquina de Estados

Minerador & Ferreiro: dois agentes com o padrão State

Inteligência Artificial e Ilusão de Inteligência em Jogos · Prof.^a Marina de Lara · PUCPR
Nome do(a) aluno(a) · Outubro de 2026

1. Visão geral do cenário

O cenário é uma pequena mina de uma vila medieval, simulada em turnos discretos (ticks). Dois personagens não-jogáveis convivem nela: um **Minerador**, que passa o dia extraindo minério, levando a carga até o depósito e parando para descansar quando se cansa, e um **Ferreiro**, que mantém a forja acesa e conserta ferramentas. A picareta do Minerador se desgasta a cada golpe; quando ela quebra, ele precisa levá-la ao Ferreiro e esperar o conserto, o que cria uma dependência natural entre os dois agentes. Todo o comportamento é determinístico, baseado em contadores e limiares simples, e cada mudança de estado é registrada no console, de modo que quem observa a execução percebe uma rotina crível — a “ilusão de inteligência” — sem nenhum elemento aleatório.

2. Visão geral dos agentes

Agente A — Minerador (4 estados). Começa em Minerando, onde cada tick extrai uma unidade de minério, gasta 10 de energia e 1 de durabilidade da picareta. Três limiares disputam a saída desse estado, em ordem de prioridade: picareta quebrada leva a AguardandoPicareta; mochila cheia leva a EntregandoMinerio; energia baixa leva a Descansando. A entrega leva dois ticks de caminhada; o descanso recupera 20 de energia por tick até o máximo. Em AguardandoPicareta o agente envia uma mensagem ao Ferreiro e fica parado até receber a resposta.

Agente B — Ferreiro (2 estados). Fica em Ocioso cuidando da forja até receber a mensagem PICARETA_QUEBRADA. Então passa a ConsertandoPicareta, onde martela a peça durante três ticks. Ao sair desse estado (método leave), devolve a picareta enviando PICARETA_CONSERTADA e volta a ficar ocioso. **Comunicação (bônus):** o estado AguardandoPicareta do Minerador só termina quando o estado ConsertandoPicareta do Ferreiro é finalizado — exatamente o tipo de dependência pedido no enunciado.

3. Diagramas de estados

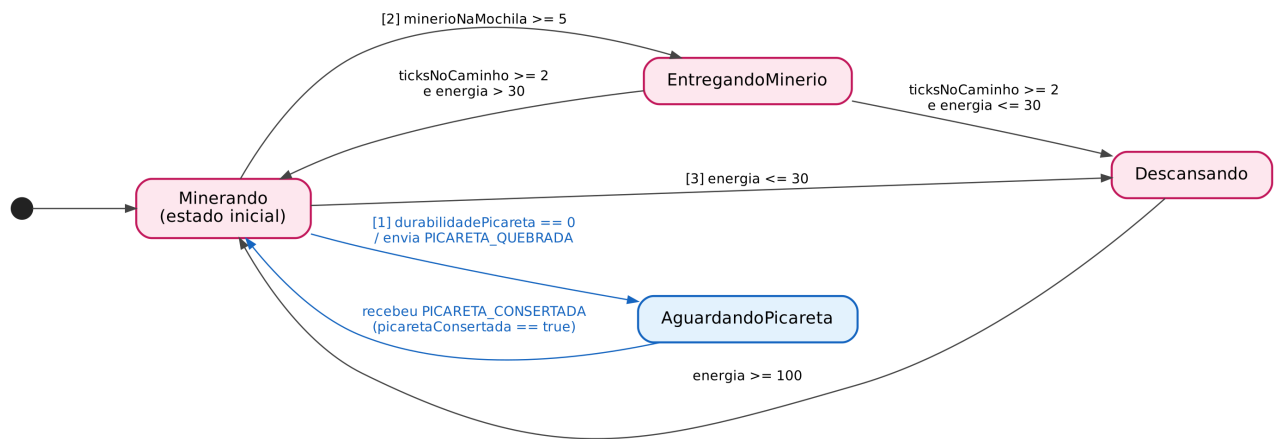


Figura 1 — Agente A (Minerador). Os números [1]-[3] indicam a prioridade das transições de Minerando; em azul, as transições ligadas à comunicação.

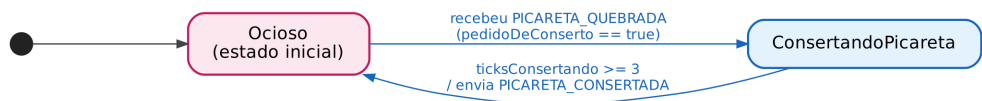


Figura 2 — Agente B (Ferreiro).

4. Tabelas de regras

4.1 Minerador

Estado	Entrada (enter)	Execução (execute)	Saída (leave)	Transições
Minerando	Log: desce para a galeria.	energia -10; durabilidade -1; mochila +1; log com status.	Log: guarda a picareta.	[1] durabilidade == 0 → AguardandoPicareta [2] mochila ≥ 5 → EntregandoMinerio [3] energia ≤ 30 → Descansando
EntregandoMinerio	ticksNoCaminho = 0; log.	ticksNoCaminho +1; ao chegar a 2, deposita a mochila.	Log: sai do depósito.	ticks ≥ 2 e energia ≤ 30 → Descansando ticks ≥ 2 e energia > 30 → Minerando
Descansando	Log: senta para descansar.	energia +20 (máx. 100); log com status.	Log: levanta renovado.	energia ≥ 100 → Minerando
Aguardando-Picareta	Log; envia PICARETA_QUEBRADA ao Ferreiro.	Se picaretaConsertada: durabilidade = 8, flag = false. Senão, log de espera.	Log: volta para a mina.	picaretaConsertada == true → Minerando

4.2 Ferreiro

Estado	Entrada (enter)	Execução (execute)	Saída (leave)	Transições
Ocioso	ticksConsertando = 0; log: abre a ferraria.	Log: mantém a forja acesa.	Log: cliente chegou.	pedidoDeConserto == true → ConsertandoPicareta
Consertando-Picareta	pedidoDeConserto = false; ticksConsertando = 0; log.	ticksConsertando +1; log do progresso.	Log; envia PICARETA_CONSERTADA ao Minerador.	ticksConsertando ≥ 3 → Ocioso

5. Variáveis e limiares

5.1 Minerador

Nome	Tipo	Intervalo	Inicial	Uso / limiar de troca
energia	int	0 - 100	100	-10 por tick minerando, +20 descansando. ≤ 30 → Descansando; ≥ 100 → Minerando.
minerioNaMochila	int	0 - 5	0	+1 por tick minerando. ≥ 5 (CAPACIDADE_MOCHILA) → EntregandoMinerio.
durabilidadePicareta	int	0 - 8	8	-1 por golpe. == 0 → AguardandoPicareta; volta a 8 após o conserto.
ticksNoCaminho	int	0 - 2	0	Timer da entrega. ≥ 2 (TICKS_PARA_ENTREGAR) → deposita e sai do estado.
minerioDepositado	int	0 - ∞	0	Total entregue (apenas informativo no log).
picaretaConsertada	boolean	true/false	false	Flag ligada pela mensagem PICARETA_CONSERTADA. true → Minerando.

5.2 Ferreiro

Nome	Tipo	Intervalo	Inicial	Uso / limiar de troca
pedidoDeConserto	boolean	true/false	false	Flag ligada pela mensagem PICARETA_QUEBRADA. true → ConsertandoPicareta.
ticksConsertando	int	0 - 3	0	Timer do conserto. ≥ 3 (TICKS_PARA_CONSERTAR) → Ocioso.
picaretasConsertadas	int	0 - ∞	0	Contador informativo exibido no log.
clienteAtual	Character	—	null	Quem pediu o conserto; destinatário da resposta.

6. Estrutura do código

O projeto segue a arquitetura de FSM genérica vista em aula (exemplo Juca/Bob): uma interface `State<C>`, uma classe abstrata `AbstractState<C>`, a interface `Character` implementada por cada agente e a classe `StateMachine` com a lista de personagens e o loop principal. Todos os arquivos ficam em `src/`.

Arquivo / classe	Papel no projeto
State<C>	Interface genérica do padrão State: <code>getCharacter()</code> , <code>enter()</code> , <code>execute()</code> e <code>leave()</code> .

Arquivo / classe	Papel no projeto
AbstractState<C>	Guarda a referência ao personagem e implementa enter() e leave() vazios; todos os estados herdam dela.
Character	Interface dos agentes: update(), setState(), printStats() e receiveMessage() (acrescentado para a comunicação).
StateMachine	Classe responsável pela execução: adiciona os agentes à lista characters e, no loop principal , chama update() de cada um a cada tick. Contém o main.
Mensagem	Enum com PICARETA_QUEBRADA e PICARETA_CONSERTADA.
Minerador	Agente A (implementa Character): variáveis, limiares, gets & adds, envio do pedido de conserto. setState() chama leave(), imprime a transição, troca o estado e chama enter().
Minerando, EntregandoMinerio, Descansando, AguardandoPicareta	Estados do Minerador; cada um herda de AbstractState<Minerador>.
Ferreiro	Agente B (implementa Character): flag de pedido, contador de conserto e envio da resposta.
Ocioso, ConsertandoPicareta	Estados do Ferreiro; herdam de AbstractState<Ferreiro>.

Como no exemplo da aula, cada transição cria a instância do próximo estado (ex.: new Descansando(getCharacter())) e as regras de transição ficam no execute() de cada estado.

7. Resultados esperados

Com os valores padrão, em 30 ticks o Minerador percorre todos os seus estados: enche a mochila e entrega (ticks 5–7), cansa e descansa (9–13), quebra a picareta e espera o Ferreiro (14–18) e repete o ciclo. Trecho real da execução:

```

===== TICK 5 =====
[MINERADOR] Minerando... | Energia: 50 | Mochila: 5/5 | Picareta: 3/8 | Depositado: 0
[MINERADOR] Para de minerar e guarda a picareta no cinto.
[MINERADOR] >>> TRANSICAO: Minerando -> EntregandoMinerio
[MINERADOR] Mochila cheia! Vai em direcao ao deposito.
...
===== TICK 14 =====
[MINERADOR] Minerando... | Energia: 90 | Mochila: 3/5 | Picareta: 0/8 | Depositado: 5
[MINERADOR] Para de minerar e guarda a picareta no cinto.
[MINERADOR] >>> TRANSICAO: Minerando -> AguardandoPicareta
[MINERADOR] A picareta quebrou! Leva a picareta ao Ferreiro.
[MINERADOR] *** MENSAGEM para o Ferreiro: PICARETA_QUEBRADA
[FERREIRO] Um cliente chegou! Larga o que estava fazendo.
[FERREIRO] >>> TRANSICAO: Ocioso -> ConsertandoPicareta
[FERREIRO] Recebe a picareta quebrada e aquece o metal na forja.
...
===== TICK 17 =====
[MINERADOR] Esperando o Ferreiro... | Energia: 90 | Mochila: 3/5 | Picareta: 0/8 | Depositado: 5
[FERREIRO] Martelando a picareta... | Conserto: 3/3 | Picaretas consertadas: 0
[FERREIRO] Resfria a picareta na agua e devolve ao Minerador.
[FERREIRO] *** MENSAGEM para o Minerador: PICARETA_CONSERTADA
[FERREIRO] >>> TRANSICAO: ConsertandoPicareta -> Ocioso
[FERREIRO] Abre a ferraria e aguarda clientes.
===== TICK 18 =====
[MINERADOR] Recebeu a picareta! | Energia: 90 | Mochila: 3/5 | Picareta: 8/8 | Depositado: 5
[MINERADOR] Agradece ao Ferreiro e volta para a mina.
[MINERADOR] >>> TRANSICAO: AguardandoPicareta -> Minerando
[MINERADOR] Desce para a galeria e empunha a picareta.
[FERREIRO] Ocioso... | Conserto: 0/3 | Picaretas consertadas: 1

```

Cada troca aparece como a sequência leave do estado antigo → linha **>>> TRANSICAO** → enter do novo estado. As linhas ***** MENSAGEM** mostram a comunicação: o Ferreiro sai de Ocioso no mesmo tick em que recebe o pedido, e o Minerador volta a minerar no tick seguinte à resposta.

8. Limitações

As mensagens são entregues de forma imediata e síncrona, por chamada direta a receiveMessage(), sem fila nem atraso; uma fila de mensagens com tempo de entrega deixaria a comunicação mais realista. O Ferreiro atende apenas um cliente por vez — se houvesse vários mineradores, um segundo pedido durante um conserto seria absorvido pela mesma flag. A simulação é somente em console, sem representação visual da mina.

9. Referências

BUCKLAND, M. Programming Game AI by Example. Wordware Publishing, 2005 — cap. 2 (máquinas de estados finitos e o exemplo do minerador), base do modelo enter/execute/exit.

GAMMA, E. et al. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994 — padrão State. Material de aula da disciplina: Evoluindo a Máquina de Estados: de Juca para Múltiplos Agentes e código FSMGenerics-MultipleAgents.